A Reproducible Empirical Comparison of SAT Encodings and Exact Solvers for the N-Queens Problem

Nguyen Anh Son

University of Engineering and Technology, Vietnam National University, Hanoi

ARTICLE INFO

Keywords:
N-Queens
Boolean Satisfiability
Constraint Programming
Mixed Integer Programming
CDCL Heuristics
Empirical Benchmarking

ABSTRACT

The N-Queens problem is a classic combinatorial benchmark widely used to evaluate algorithmic efficiency in artificial intelligence. This study presents a rigorous empirical comparison of exact solving paradigms—Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP). We evaluate five Boolean SAT encodings alongside four exact solvers within a strict, subprocess-isolated, deterministically seeded benchmarking framework. Our primary full benchmark covers up to $N = 100$ (495 records), supplemented by a large-scale experiment at $N = 200$.

Our results demonstrate that structural compactness in SAT encodings is a poor predictor of Conflict-Driven Clause Learning (CDCL) resolution time; the SAT-Sequential encoding generated the second-fewest clauses but timed out at $N = 200$, whereas heavily parameterized SAT formulations resolved much faster. At $N = 200$, SAT-Commander and SAT-Binary successfully completed the benchmark within a second, whereas SAT-Pairwise was excluded due to hardware memory limits (> 4GB for 13.2 million clauses). Notably, the SAT-Product encoding exhibited extreme non-monotonic search pathology, timing out at intermediate problem sizes while resolving $N = 100$ and $N = 200$ successfully. Constraint Programming offers a compact high-level formulation using N integer decision variables and global constraints, avoiding the need to explicitly construct the SAT CNF encodings evaluated in this study. However, the observed memory consumption depends on the solver implementation and does not consistently favor CP across all tested configurations. At $N = 200$, IBM CP Optimizer's Community Edition hit a hard search-space license limit (2^{1000}), emphasizing that both algorithmic complexity and software environment restrictions dictate practical large-scale solvability. This paper provides a fully reproducible dataset and quantitative insights into the practical trade-offs of constraint modeling techniques.

1. Introduction

The Constraint Satisfaction Problem (CSP) framework is foundational to modeling complex combinatorial search and placement problems in computer science and artificial intelligence (Russell and Norvig, 2010). Within this domain, the N-Queens problem—the challenge of placing N non-attacking queens on an $N \times N$ chessboard—serves as a canonical benchmark for evaluating the efficiency and scalability of exact search algorithms (Bell and Stevens, 2009). As the problem size N increases, the state space expands exponentially, rendering naive enumeration computationally intractable and necessitating sophisticated algorithmic paradigms.

Currently, exact solvers rely on three primary modeling paradigms: Boolean Satisfiability (SAT), Constraint Programming (CP), and Mixed Integer Programming (MIP) (Gomes, Kautz, Sabharwal and Selman, 2008; Rossi, Van Beek and Walsh, 2006; Wolsey, 2020). In the Boolean SAT paradigm, high-level structural constraints must be transformed into Conjunctive Normal Form (CNF) via variable-heavy At-Most-One (AMO) encodings. Different encoding strategies—such as Pairwise, Binary, Sequential Counter, Commander, and Product—drastically alter the size and structure of the resulting CNF formula (Sinz, 2005; Frisch and Giannaros, 2005). Conversely, native Constraint Programming implementations maintain the problem's global structural semantics by directly applying constraints over integer variables (Régin, 1994), while Mixed Integer Programming leverages linear inequality relaxations over binary decision bounds.

A recurring assumption in SAT modeling is that structural complexity—specifically the total number of generated clauses or auxiliary variables—correlates directly with the time required by Conflict-Driven Clause Learning (CDCL) solvers to find a solution. However, real-world benchmarking often reveals non-linear scaling behaviors heavily influenced by the solver's internal heuristics (Marques-Silva, Lynce and Malik, 2009). Furthermore, while algorithmic

complexity theoretically bounds performance, the practical runtime characteristics across the SAT, CP, and MIP paradigms on the exact same problem formulations remain sparsely compared in unified experimental setups.

The objective of this research is to execute a strictly controlled empirical comparison of these exact solving techniques on the N-Queens problem up to $N = 100$. Specifically, we address the following research questions: 1. How do different AMO encodings affect the structural complexity of SAT formulations? 2. How does the choice of SAT encoding impact solving time and completion reliability under a uniform CDCL configuration? 3. How do general-purpose Boolean reasoning frameworks compare against specialized native CP and MIP solvers? 4. What are the practical scalability limitations of these paradigms?

To answer these questions, we constructed an automated, subprocess-isolated benchmarking framework. We evaluate five SAT encodings solved via PySAT Glucose3, against OR-Tools CP-SAT, IBM CP Optimizer, Gurobi MIP, and IBM CPLEX MIP.

The primary contributions of this paper are:

- A comprehensive, reproducible empirical dataset reporting 540 scheduled experimental records (495 at $N \leq 100$ and 45 at $N = 200$) across 9 distinct exact solving methodologies, including 450 successfully validated solutions and explicitly documented timeout, licensing, and resource-policy outcomes.
- Evidence that structural compactness (clause count minimization) is a poor predictor of CDCL resolution time on placement constraints.
- The identification of severe non-monotonic search pathologies in specific structural encodings (e.g., the SAT-Product encoding) under standard phase-selection heuristics.
- A direct comparative baseline demonstrating the scalability advantages of native Constraint Programming ($O(N)$ formulation) while highlighting specific SAT encodings that remain highly competitive at scale.

2. Background and Related Work

2.1. Propositional Satisfiability and CDCL

Propositional Satisfiability (SAT) is the problem of determining if there exists an assignment of boolean values to variables that satisfies a given formula. The Boolean satisfiability problem was the first known NP-complete problem (Cook, 1971). Most modern SAT solvers require the input to be in Conjunctive Normal Form (CNF), meaning the formula is a conjunction of clauses, where each clause is a disjunction of literals. Modern exact SAT solving is dominated by Conflict-Driven Clause Learning (CDCL) algorithms (Marques-Silva et al., 2009), which extend the classic DPLL backtracking search by inferring and learning new conflict clauses to prune the search space dynamically. Solvers such as Glucose heavily optimize this process by aggressively managing the learned clause database based on Literal Block Distance (LBD) (Audemard and Simon, 2009).

2.2. At-Most-One (AMO) Encodings

Translating cardinality constraints—such as "at most one variable among a set can be true"—into CNF is a critical aspect of SAT modeling. The naive **Pairwise** (or Binomial) encoding operates without auxiliary variables by generating an exclusion clause for every possible pair, requiring $O(m^2)$ clauses for a group of m variables (Frisch and Giannaros, 2005). To mitigate the quadratic clause explosion, auxiliary variables are introduced. The **Binary** (or Bitwise) encoding maps the m variables to $\lceil \log_2 m \rceil$ auxiliary bits, generating $O(m \log m)$ clauses (Frisch and Giannaros, 2005). The **Sequential Counter** encoding by Sinz (2005) mimics a unary adder, bounding both auxiliary variables and clauses to $O(m)$. The **Commander** encoding (Klieber, Kwon and Rogers, 2007) recursively partitions the original variables into small groups, electing a "commander" variable for each group and generating $O(m)$ auxiliary variables and clauses. Finally, the **2-Product** encoding structures variables onto a 2D grid, translating the 1D AMO constraint into row and column constraints using $O(\sqrt{m})$ auxiliary variables (Chen, 2010).

2.3. Constraint Programming and Mixed Integer Programming

Constraint Programming (CP) provides a higher-level abstraction than SAT by operating directly on integer domains (Rossi et al., 2006). Central to CP is the `AllDifferent` global constraint, which ensures that all variables in a given set take distinct values. Efficient domain filtering algorithms, notably those based on bipartite matching (Régin, 1994), actively prune the integer domains during search without needing to expand the logic into millions of binary

clauses. Tools like OR-Tools CP-SAT (Perron and Furnon, 2019) and IBM CP Optimizer leverage these techniques heavily. Mixed Integer Programming (MIP) relies on linear inequalities over continuous and integer variables (Wolsey, 2020). While powerful for optimization problems, applying MIP solvers like Gurobi or CPLEX to pure feasibility problems like N-Queens translates logical implications into algebraic constraints bounded by linear programming relaxations.

3. Methodology

3.1. Mathematical Definition of N-Queens

The N-Queens problem requires placing N queens on an $N \times N$ chessboard such that no two queens attack each other. In the Boolean paradigm, the problem is modeled using N^2 primary decision variables $x_{i,j} \in \{0, 1\}$, where:

$$x_{i,j} = \begin{cases} 1 & \text{if a queen occupies row } i \text{ and column } j \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

for $0 \leq i, j < N$.

This formulation strictly enforces four directional constraints:

- **Row Constraints:** Exactly one queen per row.

$$\sum_{j=0}^{N-1} x_{i,j} = 1 \quad \forall i \in \{0, \dots, N-1\} \tag{2}$$

- **Column Constraints:** Exactly one queen per column.

$$\sum_{i=0}^{N-1} x_{i,j} = 1 \quad \forall j \in \{0, \dots, N-1\} \tag{3}$$

- **Main Diagonal Constraints:** At most one queen on any main diagonal. Let $d = i - j$. For each $d \in \{-(N-2), \dots, N-2\}$:

$$\sum_{i-j=d} x_{i,j} \leq 1 \tag{4}$$

- **Anti-Diagonal Constraints:** At most one queen on any anti-diagonal. Let $s = i+j$. For each $s \in \{1, \dots, 2N-3\}$:

$$\sum_{i+j=s} x_{i,j} \leq 1 \tag{5}$$

Note that diagonals with only a single cell ($d \in \{-(N-1), N-1\}$, $s \in \{0, 2N-2\}$) trivially satisfy the at-most-one condition and do not require explicit constraints.

3.2. Boolean SAT Constraints and Encodings

In Boolean Satisfiability, an *Exactly-One* (EO) constraint is represented as the logical conjunction of an *At-Least-One* (ALO) clause and an *At-Most-One* (AMO) constraint:

$$EO(X) = ALO(X) \wedge AMO(X) \tag{6}$$

For a set of boolean variables $X = \{x_1, \dots, x_m\}$, the ALO constraint is simply a single disjunction:

$$ALO(x_1, \dots, x_m) = \bigvee_{k=1}^{m} x_k \tag{7}$$

The AMO constraint, however, can be encoded into Conjunctive Normal Form (CNF) using various strategies. For instance, the Pairwise AMO encoding asserts that no two variables can be simultaneously true:

$$AMO_{Pairwise}(x_1, \dots, x_m) = \bigwedge_{1 \le a < b \le m} (\neg x_a \vee \neg x_b) \tag{8}$$

Example: For a small set of 4 Boolean variables $\{x_1, x_2, x_3, x_4\}$, enforcing Exactly-One using Pairwise AMO requires one ALO clause and six AMO clauses:

- ALO: $(x_1 \vee x_2 \vee x_3 \vee x_4)$
- AMO: $(\neg x_1 \vee \neg x_2) \wedge (\neg x_1 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_4) \wedge (\neg x_2 \vee \neg x_3) \wedge (\neg x_2 \vee \neg x_4) \wedge (\neg x_3 \vee \neg x_4)$

While all AMO encodings preserve the mathematical validity of the N-Queens problem, they differ fundamentally in how they represent AMO structurally. Our framework implements five precise variants of AMO encodings:

1. **Pairwise**: Asserts $\neg x_a \vee \neg x_b$ for all pairs a, b. No auxiliary variables are used. Generates $\Theta(m^2)$ clauses for a group of m literals. Summed over all groups, the entire N-Queens formulation yields $\Theta(N^3)$ clauses.
2. **Binary**: Each original variable forces a unique binary combination on $k = \lceil \log_2 m \rceil$ shared auxiliary bits. Generates $O(\log m)$ auxiliary variables and $O(m \log m)$ clauses per group.
3. **Sequential Counter**: Applies Sinz's formulation (Sinz, 2005), establishing a chain of $m - 1$ auxiliary variables to cumulatively count the number of true literals, forbidding the count from exceeding one. Generates $O(m)$ auxiliary variables and $O(m)$ clauses per group.
4. **Commander**: Recursively groups variables into sets of three (Frisch and Giannaros, 2005). If any variable in a subgroup is true, its commander auxiliary is true, and Pairwise AMO is enforced locally. The complexity depends on the grouping strategy.
5. **2-Product**: Maps m variables onto a $p \times q$ grid where $p \approx \lceil \sqrt{m} \rceil$. Each variable forces one auxiliary row and one auxiliary column to be true. Pairwise AMO is then applied to the auxiliary rows and columns independently.

All SAT formulations are generated via the Python API and passed to the Glucose3 solver (Audemard and Simon, 2009) via the PySAT library. To ensure strict isolation of structural efficiency from heuristic tuning, the `solver_default` phase policy was enforced.

3.3. Comparison of Mathematical Formulations and Solving Paradigms

Our evaluation directly compares three fundamentally different exact solving paradigms applied to the exact same instance sizes of the N-Queens problem.

Boolean SAT: The problem is modeled with N^2 primary Boolean variables and purely logical CNF constraints. We evaluate the five AMO encodings (Pairwise, Binary, Sequential, Commander, Product), all solved via the CDCL-based Glucose3 solver with a common phase policy.

Constraint Programming (CP): CP frameworks maintain the problem's high-level semantics natively without Boolean expansion. Using N integer decision variables $q_i \in \{0, \dots, N-1\}$ (representing the column of the queen in row i), the constraints are modeled as three global `AllDifferent` constraints (Régin, 1994):

$$\text{AllDifferent}(q_0, \dots, q_{N-1}) \tag{9}$$

$$\text{AllDifferent}(q_0 - 0, \dots, q_{N-1} - (N-1)) \tag{10}$$

$$\text{AllDifferent}(q_0 + 0, \dots, q_{N-1} + (N-1)) \tag{11}$$

This formulation was solved independently using OR-Tools CP-SAT and IBM CP Optimizer. While the formulation is identical, the internal solving algorithms and heuristics differ between the two solvers.

Mixed Integer Programming (ILP/MIP): Using the same N^2 binary decision variables $x_{i,j} \in \{0, 1\}$ as the SAT model, the MIP formulation translates the logical constraints into continuous linear equality and inequality constraints.

Table 1
Mathematical Formulations of Exact Solving Paradigms

Paradigm	Primary Variables	Constraints
SAT	$O(N^2)$ Boolean	$O(N^2)$ to $\Theta(N^3)$ clauses (dependent on AMO strategy)
CP	$O(N)$ Integer	3 Global `AllDifferent` constraints
MIP	$O(N^2)$ Binary	$6N - 6$ Linear equality and inequality constraints

Table 2
Experimental Configuration

Parameter	Value
OS	macOS 14.0 (Darwin 23.0.0, ARM64)
Python Version	3.13.7
SAT Phase Policy	`solver_default`
Time Limit	300.0s (Framework/Internal), 330.0s (Subprocess Watchdog)
Workers	1
Random Seed	0
Repetitions	5 per configuration

For $N \geq 2$, it requires $6N - 6$ constraints: $2N$ equality constraints for rows and columns, and $4N - 6$ inequality constraints for the diagonals. The model is formulated as a 0-1 integer programming feasibility problem (no objective function) and solved using Gurobi MIP and IBM CPLEX MIP.

Experimental Fairness: To ensure an unbiased comparative baseline, all paradigms are subject to strict experimental fairness parameters. They solve identical N-Queens instance sizes searching for any single valid solution. We enforce single-threaded execution when supported by the solver architecture, apply the same time limits, execute inside the same hardware/software benchmarking environment, and repeat each configuration 5 times. Importantly, all solutions returned by any solver are mathematically verified by a common external validator to guarantee structural correctness. Note that while the formulations are mathematically equivalent, their internal solver representations and search heuristics are entirely distinct.

4. Experimental Setup

The benchmark was conducted on a controlled hardware and software environment to ensure deterministic execution. The experiments were run on macOS 14.0 (Darwin 23.0.0, Apple M2 ARM64 architecture) using Python 3.13.7. The evaluated engines included PySAT Glucose3, OR-Tools CP-SAT (version 9.15.6755), Gurobi Optimizer (version 13.0.3), and IBM ILOG CPLEX / CP Optimizer (version 22.2.0.0).

We evaluated 9 distinct solving methods across 11 instance sizes: $N \in \{4, 8, 16, 20, 31, 32, 40, 44, 50, 64, 100\}$. Each configuration was repeated 5 times, yielding 495 planned official trials. To guarantee fairness, every trial was executed in an isolated subprocess. All methods were strictly configured to use single-threaded execution (`workers=1`, `threads=1`) with a deterministic global random seed (`random_seed=0`). A primary timeout of 300.0 seconds was enforced by the external benchmarking framework, supplemented by a 330.0-second operating system subprocess kill signal (watchdog) to forcefully terminate solvers that failed to yield control. For solvers exposing a native API time limit (e.g., Gurobi, CPLEX, CP-SAT), the internal limit was also set to 300.0 seconds; otherwise (e.g., PySAT Glucose3), execution was strictly managed by the framework's external watchdog.

The primary metric for performance evaluation is **pipeline total time**, which comprehensively captures the end-to-end framework execution time, including model construction, solver instantiation, search, and final independent validation. Runs blocked by proactive Community Edition license constraints (BLOCKED_LICENSE) were explicitly recorded to differentiate API capacity limitations from algorithmic failures.

5. Results and Discussion

Across the 495 planned trials, 430 yielded successfully verified solutions, 20 encountered timeouts, and 45 were blocked by expected MIP license limitations. No solver generated an invalid placement.

5.1. Structural Comparison of SAT Encodings

As instance size grows, the structural disparities of CNF generation become stark. At $N = 100$, all encodings instantiate identical 10,000 primary binary variables but vary widely in auxiliary metrics. The Pairwise encoding requires zero auxiliary variables but scales poorly, producing 1,646,900 clauses. The Sequential Counter encoding generates the second-fewest clauses (117,812) at the cost of the highest auxiliary count (39,402). The Product encoding achieves the most compact configuration, producing 116,672 clauses and 9,492 auxiliary variables. Binary and Commander encodings balance these metrics.

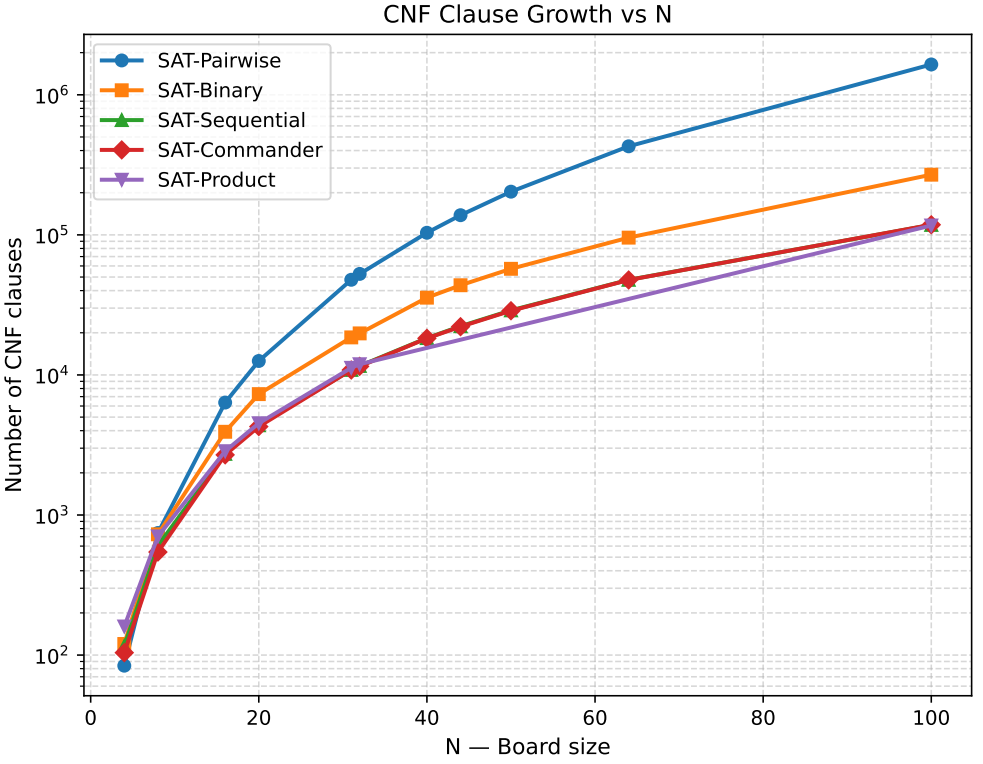


Figure 1: SAT CNF Clause Growth

5.2. Runtime Performance of SAT Encodings

The resolution time for SAT formulations proves that clause count is an inadequate predictor of performance.

At $N = 100$, the SAT-Binary encoding achieved the fastest median pipeline runtime of 0.231 seconds. In stark contrast, the SAT-Sequential encoding, despite producing minimal clauses, recorded the slowest median runtime of 58.077 seconds. The clause-heavy SAT-Pairwise encoding successfully completed the search in a median time of 1.148 seconds.

5.3. Non-monotonic Search Behavior of Product Encoding

While SAT-Product reliably solved small sizes and successfully resolved $N = 100$ (median 0.620s), it completely failed at intermediate sizes ($N \in \{40, 44, 50, 64\}$), resulting in a timeout for all 5 repetitions.

This non-monotonic scaling underscores the sensitivity of CDCL solvers to heuristic traps. Under Glucose3's `solver_default` policy, the specific auxiliary-variable structure of Product at intermediate bounds may force the search

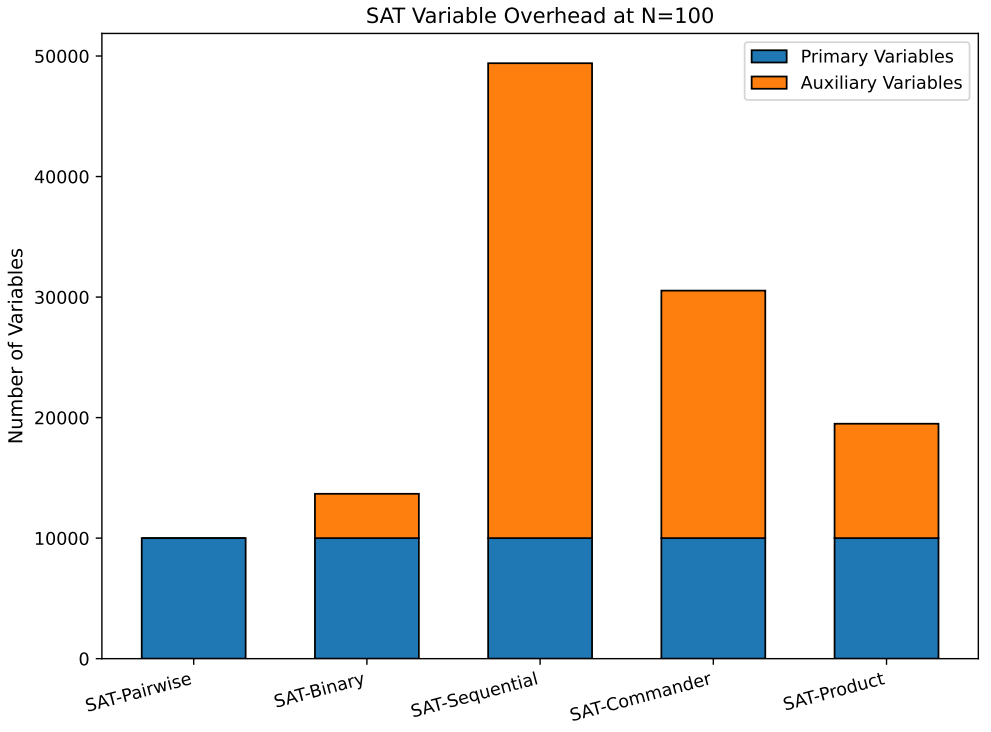


Figure 2: SAT Auxiliary Variable Overhead

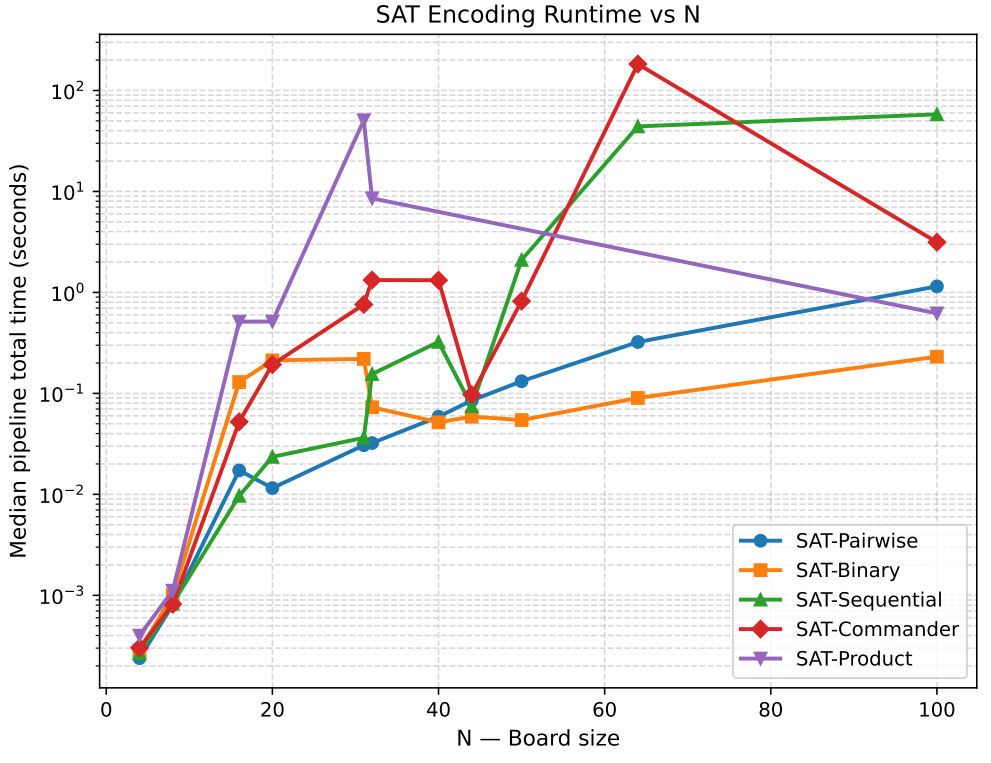


Figure 3: SAT Runtime vs N

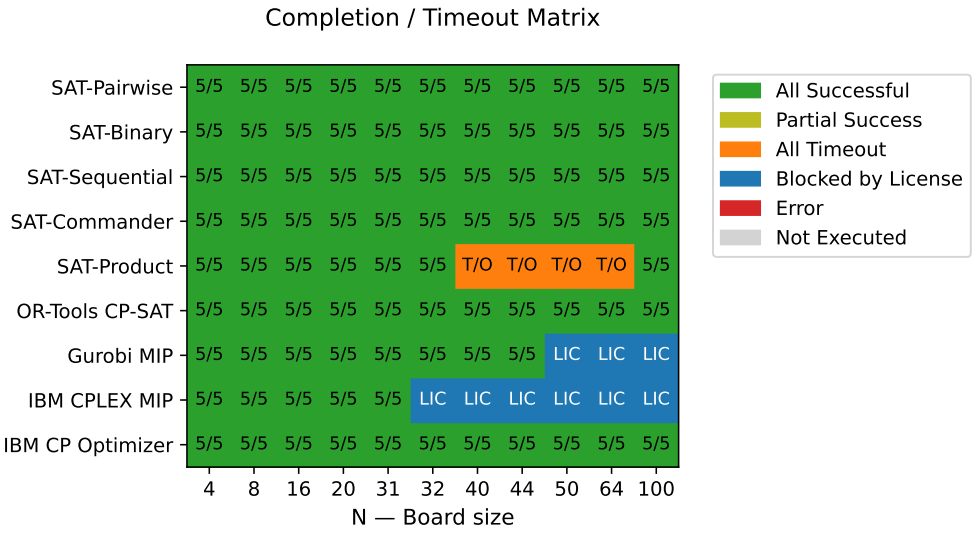


Figure 4: Completion Matrix

Table 3
SAT Encoding Structural Complexity at N=100

Encoding	Primary Variables	Auxiliary Variables	Clauses
Pairwise	10,000	0	1,646,900
Binary	10,000	3,678	269,024
Sequential	10,000	39,402	117,812
Commander	10,000	20,536	118,092
Product	10,000	9,492	116,672

into pathological branching spaces. However, these are potential contributing factors; analyzing the exact causes of this behavior requires further experimental study of internal solver traces and variable ordering.

5.4. Performance of Exact Solvers

Native Constraint Programming (CP) architectures demonstrated highly favorable scaling characteristics.

Both IBM CP Optimizer and OR-Tools CP-SAT maintained 100% reliability up to $N = 100$. The CP Optimizer solver established itself as remarkably efficient. The MIP solvers successfully solved bounds within their Community Edition license constraints.

5.5. Cross-Paradigm Comparison at N=100

At $N = 100$, seven methods successfully executed. The IBM CP Optimizer achieved the lowest median runtime of 0.153 seconds.

Remarkably, the booleanized SAT-Binary formulation tracked incredibly close at 0.231 seconds, outpacing OR-Tools CP-SAT (0.808 seconds).

6. Threats to Validity

Several factors condition the interpretation of our results. First, to establish an un-tuned algorithmic baseline, all Boolean SAT encodings were executed under the Glucose3 default phase policy. Modifying branching heuristics could radically alter search trajectories; therefore, the timeouts and non-monotonic behavior observed in the SAT-Product encoding may be artifacts of this specific heuristic interaction rather than inherent failures of the encoding space. Second, the primary metric (pipeline total time) aggregates CNF construction and search. It is critical to distinguish

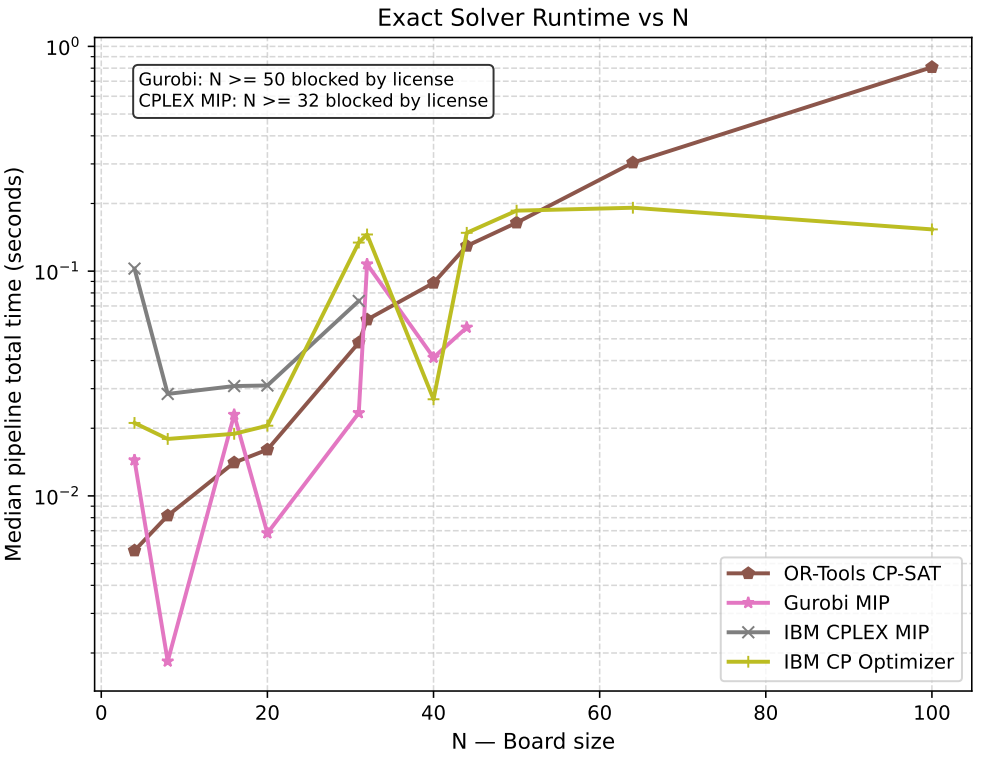


Figure 5: Exact Solver Runtime vs N

Table 4
Cross-Paradigm Comparison at N=20

Method	Success Rate	Median Runtime (s)
Gurobi MIP	5/5	0.0068
SAT-Pairwise	5/5	0.0115
OR-Tools CP-SAT	5/5	0.0160
IBM CP Optimizer	5/5	0.0205
SAT-Sequential	5/5	0.0235
IBM CPLEX MIP	5/5	0.0309
SAT-Commander	5/5	0.1928
SAT-Binary	5/5	0.2124
SAT-Product	5/5	0.5150

between per-group complexity and total formula complexity; while some encodings have $O(N^2)$ clauses per AMO group yielding a total of $\Theta(N^3)$ clauses (e.g., Pairwise), others require $O(N)$ clauses per group yielding $O(N^2)$ total clauses. Consequently, all SAT algorithms are burdened by structural initialization penalties not experienced by native $O(N)$ CP architectures.

The supplementary large-scale experiment at $N = 200$ introduces additional validity considerations. Execution was limited to only five repetitions per method, which may be insufficient to capture the full variance of highly stochastic CDCL search paths. Furthermore, heavy swap usage and strict preflight memory exclusions (`NOT_RUN_RESOURCE_POLICY`) were necessitated by the hardware constraints of the single macOS ARM64 environment. The IBM CP Optimizer's `LICENSE_ERROR` was explicitly triggered by a commercial 2^{1000} search-space bound rather than algorithmic inability. Finally, adding a single large $N = 200$ point provides valuable extreme-scale insights but is not sufficient to definitively project continuous asymptotic complexity curves across all intermediate problem sizes.

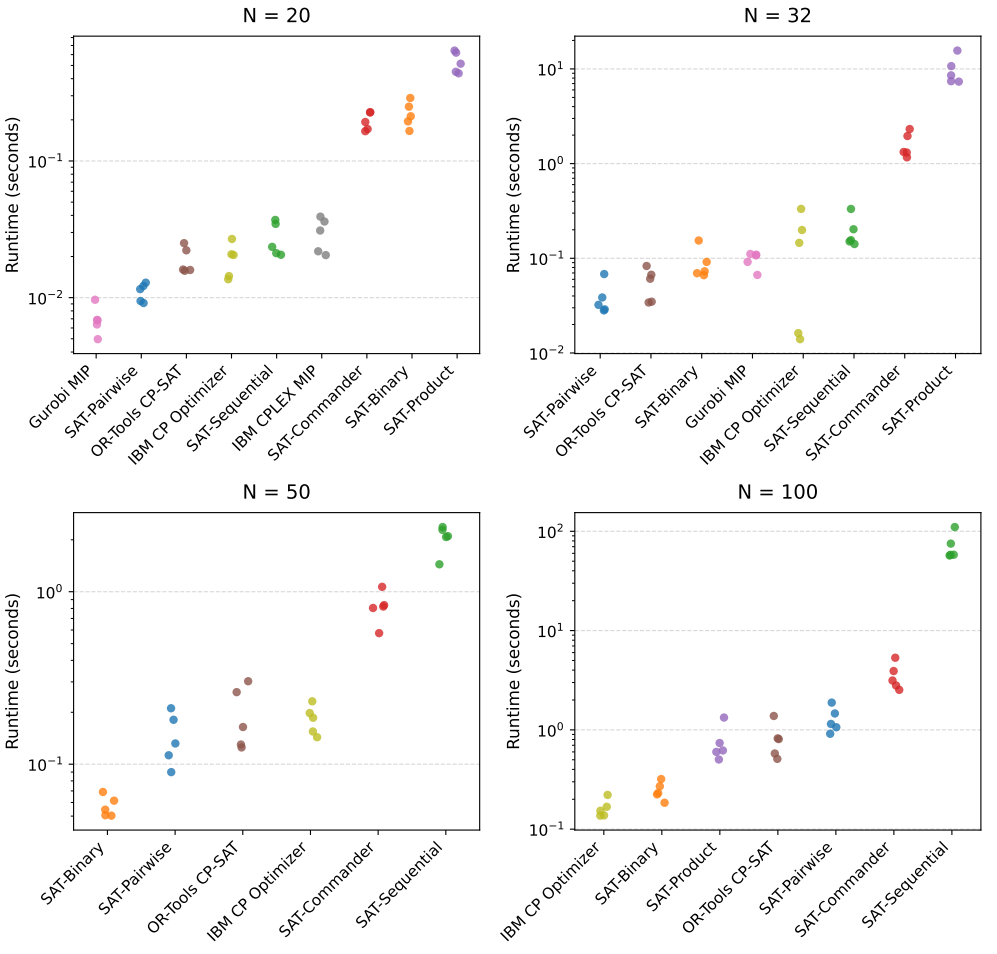


Figure 6: Runtime Variability

7. Supplementary Large-Scale Experiment

To assess the scalability of exact solving paradigms beyond the $N = 100$ boundary, an independent, supplementary experiment was conducted at $N = 200$ (representing a board size of 200×200 with 40, 000 primary Boolean variables). A strict memory guard policy was introduced to prevent hardware exhaustion. Table 7 presents the solver outcomes based on 5 isolated repetitions per method.

The scaling results fundamentally underscore the intersection of mathematical formulation and hardware capacity. The SAT-Pairwise formulation, which creates $\Theta(N^3)$ clauses, generated an estimated 13.2 million clauses at $N = 200$ requiring over 4GB of continuous memory allocation, prompting the preflight resource guard to abort execution (`NOT_RUN_RESOURCE_POLICY`). SAT-Sequential experienced universal timeouts, aligning with its severe performance degradation observed at lower values of N. Moreover, the IBM CP Optimizer encountered a `LICENSE_ERROR` triggered by its Community Edition search space limit (2^{1000} combinations), which is easily breached by the $N = 200$ variables.

As shown in Figure 8, the methods that successfully scaled (SAT-Commander, SAT-Product, SAT-Binary, and OR-Tools CP-SAT) maintained manageable memory footprints due to their structural compactness. Interestingly, the SAT-Product encoding, which previously displayed non-monotonic failures and pathological timeouts at $N = 44$ and $N = 64$, effectively recovered at $N = 200$ to achieve a median runtime of 0.627 seconds. Furthermore, SAT-Commander exhibited an inverted runtime scaling ratio (faster at $N = 200$ than $N = 100$). These observations suggest

Table 5
Cross-Paradigm Comparison at N=100

Method	Success Rate	Median Runtime (s)
IBM CP Optimizer	5/5	0.153
SAT-Binary	5/5	0.231
SAT-Product	5/5	0.620
OR-Tools CP-SAT	5/5	0.808
SAT-Pairwise	5/5	1.148
SAT-Commander	5/5	3.143
SAT-Sequential	5/5	58.077
Gurobi MIP	N/A	BLOCKED_LICENSE
IBM CPLEX MIP	N/A	BLOCKED_LICENSE

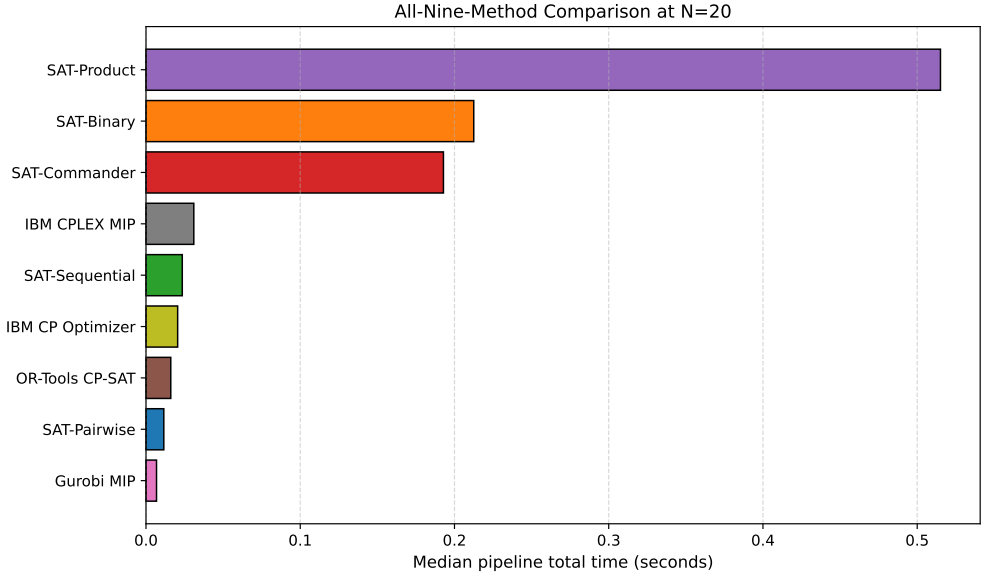


Figure 7: All Nine Methods at N=20

that specific variable partitioning strategies may favorably align with the default phase-saving heuristics of modern CDCL architectures on large symmetric grids, although confirming this requires further theoretical and empirical validation.

Figure 9 provides a holistic view of the execution outcomes at $N = 200$. While native solvers were predominantly bounded by license restrictions, SAT models demonstrated a wider variance of outcomes encompassing memory exclusions (`NOT_RUN_RESOURCE_POLICY`), timeouts, and successful resolutions.

Table 6
Completion Reliability Across All 495 Trials

Method	Successful Runs	Timeout Runs	License Blocked Runs
SAT-Pairwise	55	0	0
SAT-Binary	55	0	0
SAT-Sequential	55	0	0
SAT-Commander	55	0	0
SAT-Product	35	20	0
OR-Tools CP-SAT	55	0	0
IBM CP Optimizer	55	0	0
Gurobi MIP	40	0	15
IBM CPLEX MIP	25	0	30
Total	**430**	**20**	**45**

Table 7
Large-scale experimental outcomes at $N = 200$. Medians calculated over successful trials only.

Method	**Status**	**Success/5**	**Median Time (s)**	**Peak RAM (MB)**
SAT-Binary	SAT	5	0.815	288.9
SAT-Pairwise	NOT_RUN_RESOURCE_POLICY	0	N/A	N/A
SAT-Sequential	TIMEOUT	0	N/A	649.5
SAT-Commander	SAT	5	0.441	146.2
SAT-Product	SAT	5	0.627	165.1
OR-Tools CP-SAT	SAT	5	3.672	259.7
IBM CP Optimizer	LICENSE_ERROR	0	N/A	87.1
Gurobi MIP	BLOCKED_LICENSE	0	N/A	N/A
IBM CPLEX MIP	BLOCKED_LICENSE	0	N/A	N/A

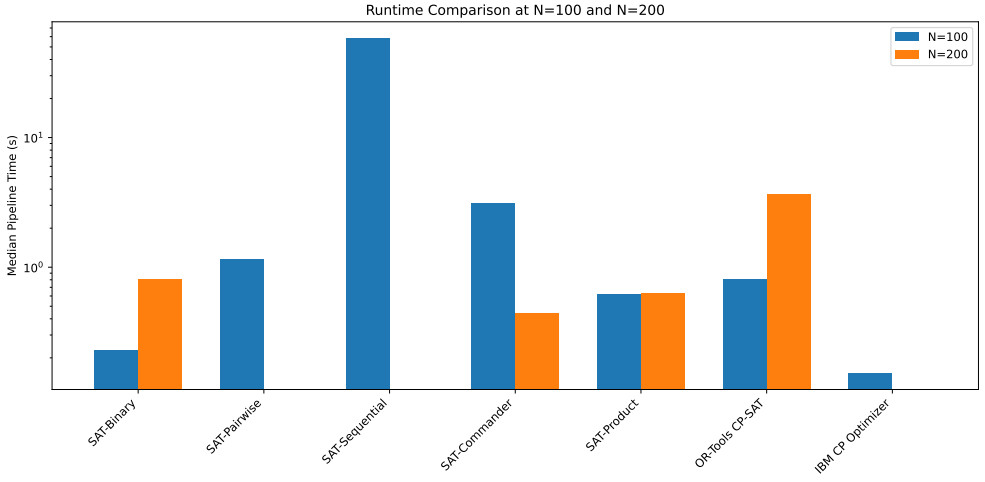


Figure 8: Median pipeline time comparison between $N = 100$ and $N = 200$ for successful methods.

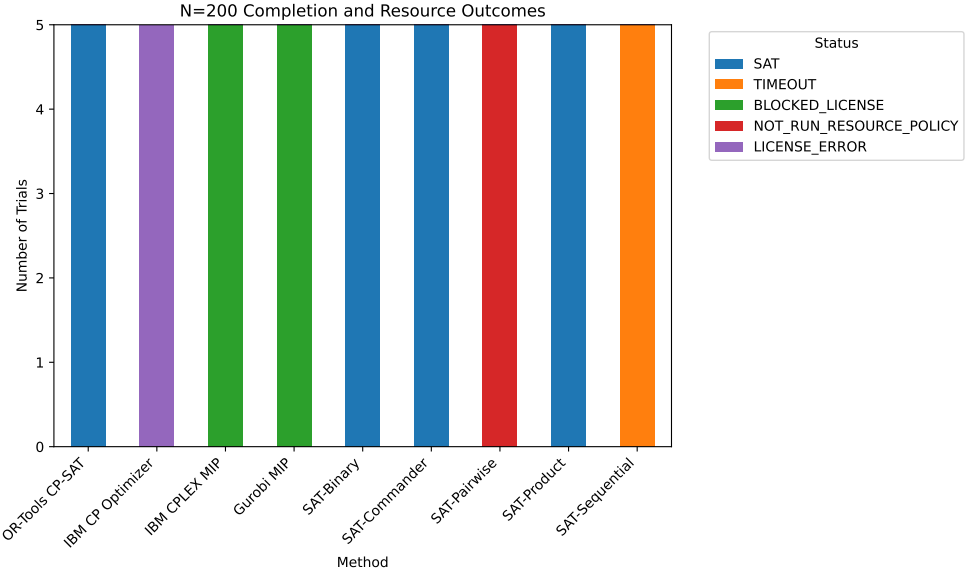


Figure 9: Distribution of completion statuses and resource limitations at $N = 200$.

8. Conclusion

This research provides a strictly controlled empirical evaluation of three major exact solving paradigms—Boolean SAT, Constraint Programming, and Mixed Integer Programming—on the N-Queens problem. Through the execution of 495 scheduled benchmark trials (430 successful completions) up to $N = 100$, we observed that theoretical SAT structural compactness is a poor predictor of practical solving efficiency. Massive clause models (Pairwise) frequently outpaced compact-clause implementations (Sequential Counter). Under default heuristic settings, the SAT-Binary encoding emerged as the most efficient boolean model, completing $N = 100$ in 0.231 seconds.

At $N = 200$, the limitations of both hardware and software environments became evident. The SAT-Pairwise encoding was excluded due to an estimated memory footprint exceeding hardware limits, while IBM CP Optimizer encountered a license-imposed search space restriction. Among the successful methods at $N = 200$, SAT-Commander achieved the fastest median resolution time of 0.441 seconds, significantly outpacing OR-Tools CP-SAT (3.672 seconds) and suggesting that optimal boolean partitioning can remain highly effective even at massive scales. However, native Constraint Programming implementations handled the large state spaces efficiently. The IBM CP Optimizer recorded the fastest runtime overall, resolving $N = 100$ in 0.153 seconds by circumventing the massive memory initialization bottlenecks required by SAT translation. Nevertheless, CP frameworks still suffer from exponential backtracks and do not universally eliminate search limitations. Furthermore, we documented severe non-monotonic search pathologies in the SAT-Product encoding, underscoring the extreme sensitivity of CDCL solvers to variable layout.

Future investigations should focus on phase-policy ablation studies to isolate the mechanisms triggering SAT search pathologies, as well as evaluating these structural encodings on a broader suite of combinatorial benchmarks beyond placement constraints.

Data Availability

The complete source code, solver implementations, test suites, and benchmarking reproduction scripts are openly available in the GitHub repository: `https://github.com/anhson1207/NQueens-SAT`. The full generated experimental datasets (including raw benchmark logs, processed CSV matrices, and statistical figures) are archived and submitted separately alongside the research documentation rather than hosted within the source-only code repository.

References

Audemard, G., Simon, L., 2009. Predicting learnt clauses quality in modern sat solvers, in: International Joint Conference on Artificial Intelligence (IJCAI).

Bell, J., Stevens, B., 2009. Solving the n-queens problem. Mathematics Magazine 82, 97–106.

Chen, J., 2010. A new sat encoding of the at-most-one constraint, in: Tenth International Workshop on Constraint Modelling and Reformulation.

Cook, S.A., 1971. The complexity of theorem-proving procedures, in: Proceedings of the third annual ACM symposium on Theory of computing, pp. 151–158.

Frisch, A.M., Giannaros, P.A., 2005. Sat encodings of the at-most-k constraint: Some old, some new, some fast, some slow, in: Workshop on Modelling and Reformulating Constraint Satisfaction Problems, pp. 23–37.

Gomes, C.P., Kautz, H., Sabharwal, A., Selman, B., 2008. Satisfiability Solvers. Elsevier.

Klieber, W., Kwon, G., Rogers, J., 2007. Solving qbf with smores, in: International Conference on Theory and Applications of Satisfiability Testing, pp. 1–7.

Marques-Silva, J., Lynce, I., Malik, S., 2009. Conflict-driven clause learning sat solvers, in: Handbook of Satisfiability, IOS Press. pp. 131–153.

Perron, L., Furnon, V., 2019. Or-tools. URL: `https://developers.google.com/optimization`.

Régin, J.C., 1994. A filtering algorithm for constraints of difference in csps, in: National Conference on Artificial Intelligence (AAAI), pp. 362–367.

Rossi, F., Van Beek, P., Walsh, T., 2006. Handbook of Constraint Programming. Elsevier.

Russell, S., Norvig, P., 2010. Artificial Intelligence: A Modern Approach. 3rd ed., Prentice Hall.

Sinz, C., 2005. Towards an optimal cnf encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming (CP), Springer. pp. 827–831.

Wolsey, L.A., 2020. Integer Programming. John Wiley & Sons.